# BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

| | |
|---|---|
| Họ và tên | Trần Đình Thi |
| MSSV / Lớp | 3123410348 |

## Mục tiêu học tập

• Mô hình hóa bài toán tìm kiếm trong AI.

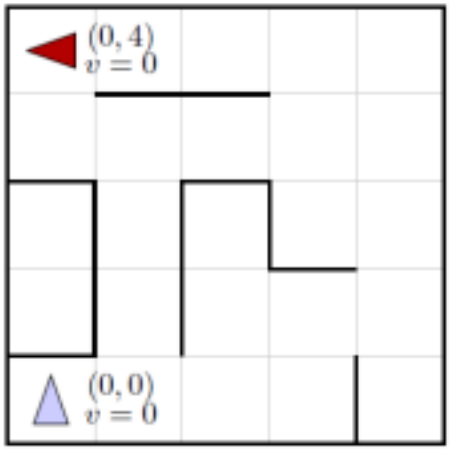
• Vận dụng BFS, UCS, Greedy và A*.

• Phân tích và đánh giá heuristic.

• Phân biệt heuristic admissible và consistent.

• So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

### Câu 1. Tìm kiếm và heuristic




Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

• Tăng tốc (Faster): v' = v + 1

• Giảm tốc (Slower): v' = v - 1

• Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

• Rẽ trái: quay 90° ngược chiều kim đồng hồ.

• Rẽ phải: quay 90° theo chiều kim đồng hồ.

**Ví dụ**: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

• Hành động làm tác tử va vào tường.

• Làm v < 0 hoặc v > Vmax.

• Giữ tốc độ khi v = 0.

• Rẽ khi v ≠ 0.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn (v = 0) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

**a)** Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Chuỗi 11 hành động ngắn nhất:
1. Rẽ phải (Right) -> tại (0,0), hướng Đông, v = 0
2. Tăng tốc (Faster) -> tiến 1 ô đến (1,0), hướng Đông, v = 1
3. Giảm tốc (Slower) -> tiến 0 ô tại (1,0), hướng Đông, v = 0
4. Rẽ trái (Left) -> tại (1,0), hướng Bắc, v = 0
5. Tăng tốc (Faster) -> tiến 1 ô đến (1,1), hướng Bắc, v = 1
6. Tăng tốc (Faster) -> tiến 2 ô đến (1,3), hướng Bắc, v = 2 (đi qua ô (1,2))
7. Giảm tốc (Slower) -> tiến 1 ô đến (1,4), hướng Bắc, v = 1
8. Giảm tốc (Slower) -> tiến 0 ô tại (1,4), hướng Bắc, v = 0
9. Rẽ trái (Left) -> tại (1,4), hướng Tây, v = 0
10. Tăng tốc (Faster) -> tiến 1 ô đến (0,4), hướng Tây, v = 1
11. Giảm tốc (Slower) -> tiến 0 ô tại (0,4), hướng Tây, v = 0 (Đích!)
Tổng chi phí: 11 hành động.

**b)** Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

Kích thước không gian trạng thái: $|S| = M \times N \times 4 \times (Vmax + 1)$
Giải thích: Mỗi trạng thái được xác định bởi bộ 4 thông số: (x, y, hướng, vận tốc).
- Vị trí (x, y): có M × N ô.
- Hướng: có 4 hướng (Bắc, Nam, Đông, Tây).
- Vận tốc v: có Vmax + 1 giá trị khả dĩ (từ 0 đến Vmax).
Theo quy tắc nhân, kích thước không gian trạng thái là $4 \times M \times N \times (Vmax + 1)$.

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☑ Có ☐ Không

Ví dụ trạng thái không có nút con:
Xét trạng thái tác tử đang có vận tốc v = 2, hướng Bắc, và ô ngay phía trước (cách 1 ô) là tường cản (hoặc mép lưới).
- Tăng tốc (v'=3, đi 3 ô): va vào tường (không hợp lệ).
- Giữ tốc độ (v'=2, đi 2 ô): va vào tường (không hợp lệ).
- Giảm tốc (v'=1, đi 1 ô): va vào tường (không hợp lệ).
- Rẽ trái/phải: không hợp lệ vì v = 2 ≠ 0 (chỉ được rẽ khi v = 0).
Do không có bất kỳ hành động nào hợp lệ, trạng thái này có 0 nút con.

**d)** Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Hệ số phân nhánh lớn nhất: 3

Trạng thái ví dụ: (x, y, Bắc, 0) ở nơi phía trước là ô trống (không có tường cản).
Các hành động có thể thực hiện (3 hành động):
1. Rẽ trái (Left)
2. Rẽ phải (Right)
3. Tăng tốc (Faster) [v' = 1, di chuyển 1 ô sang (x, y+1)]
(Hành động Giữ tốc độ không hợp lệ vì v = 0; Giảm tốc không hợp lệ vì v - 1 < 0).

Hoặc trạng thái đang di chuyển $v \in [1, Vmax - 1]$ thông thoáng phía trước có 3 hành động: Tăng tốc, Giữ tốc độ, Giảm tốc.

**e)** Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☑ Có ☐ Không

Phản ví dụ:
Giả sử Vmax ≥ 2. Xét tác tử tại vị trí (0, 1), hướng Bắc, vận tốc v = 2, và ô cửa ra tại (0, 4), hướng Bắc, v = 0.
- Giá trị Manhattan heuristic: h = |0 - 0| + |4 - 1| = 3.
- Chi phí thực tế tối ưu đến đích: Chỉ cần 2 hành động:
  1. Giảm tốc (v' = 1) -> đi 1 ô đến (0, 2), v = 1.
  2. Giảm tốc (v' = 0) -> đi 0 ô dừng tại (0, 2)? Hoặc nếu v = 3: đi 3 ô chỉ mất 1 bước!
  Cụ thể, khi tác tử đang ở vận tốc v cao, xe có thể di chuyển nhiều hơn 1 ô trong 1 bước hành động. Ví dụ xe ở (0, 0), v = 2, hướng Bắc, đích ở (0, 2), v = 2: xe chỉ cần 1 hành động Giữ tốc độ (v'=2) là đến đích (chi phí thực tế = 1), trong khi khoảng cách Manhattan là 2 (h = 2 > 1 = h*). Vì h(s) > h*(s) nên heuristic Manhattan đánh giá quá cao chi phí, KHÔNG admissible.

**f)** Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☑ Có ☐ Không

Giải thích:
Heuristic h = Manhattan / Vmax là ADMISSIBLE (chấp nhận được).
Chứng minh: Trong mỗi hành động, tác tử chỉ có thể di chuyển tối đa một khoảng cách bằng vận tốc tối đa Vmax ô. Nếu tổng khoảng cách Manhattan cần đi là D, thì với bất kỳ chuỗi hành động hợp lệ nào gồm k hành động, tổng quãng đường di chuyển tối đa không thể vượt quá k × Vmax. Do đó:
$k \times Vmax \geq D \Rightarrow k \geq D / Vmax = Manhattan / Vmax$.
Vì chi phí thực tế tối ưu h*(s) chính là số hành động k nhỏ nhất, ta có $h^*(s) \geq h(s)$ với mọi trạng thái. Vậy heuristic này không bao giờ đánh giá vượt quá chi phí thực tế, là admissible.

**g)** Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☑ Có ☐ Không

Giải thích: Trên đồ thị hữu hạn và heuristic không âm, A* Tree Search vẫn giữ tính đầy đủ (Completeness). Dù heuristic không admissible có thể đánh giá sai chi phí, thuật toán vẫn sẽ tiếp tục mở rộng các nút trong hàng đợi ưu tiên và cuối cùng sẽ duyệt tới nút đích nếu tồn tại đường đi. Do đó tính đầy đủ KHÔNG thay đổi.

**h)** Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.



☑ Có ☐ Không

Giải thích: Tính tối ưu (Optimality) CÓ THỂ thay đổi (bị mất đi). Khi heuristic không admissible (đánh giá quá cao chi phí trên đường đi tối ưu), giá trị f(n) của các nút trên đường đi tối ưu có thể lớn hơn f của một đường đi dưới tối ưu (suboptimal). A* có thể mở rộng và kết thúc tại đích theo đường đi dưới tối ưu trước, dẫn tới việc trả về nghiệm không tối ưu.

**i)** Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☑ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.
☑ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.
☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.
☑ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

☐ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn. ☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm. ☐ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

## Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

**a)** Breadth-First Search (BFS).

Bất đẳng thức: $d(n) < d(n_s)$

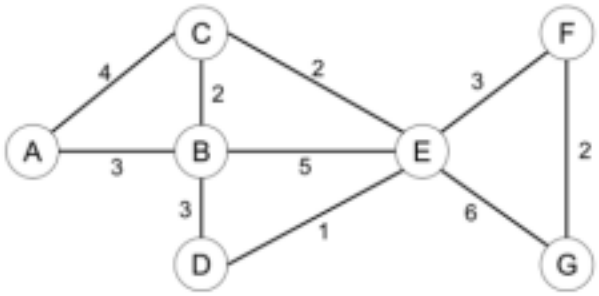
**b)** Uniform-Cost Search (UCS).

| |
|---|
| Bất đẳng thức: g(n) < g(n_c) |

**c)** A* Search, giả sử h là heuristic nhất quán.

| |
|---|
| Bất đẳng thức: g(n) + h(n) < g(n_c)  [hoặc f(n) < g(n_c)] |

## Câu 3. Tìm kiếm trên đồ thị




| Node | $h_1$ | $h_2$ |
|---|---|---|
| A | 12 | 11 |
| B | 6 | 7 |
| C | 9 | 6 |
| D | 3 | 4 |
| E | 3 | 5 |
| F | 2 | 1 |
| G | 0 | 0 |

| Nút | h1 | h2 |
|---|---|---|
| A | 12 | 11 |
| B | 6 | 7 |
| C | 9 | 6 |
| D | 3 | 4 |



| | | |
|---|---|---|
| E | 3 | 5 |
| F | 2 | 1 |
| G | 0 | 0 |

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

**a)** Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

| Thuật toán | A-C-E-G | A-C-E-F-G | A-B-D-E-F-G | Khác |
|---|---|---|---|---|
| UCS | ☐ | ☑ | ☐ | ☐ |

| | | | | |
|---|---|---|---|---|
| Greedy với h1 | ☐ | ☐ | ☐ | ☑ (A-B-D-E-G / A-B-E-G) |
| Greedy với h2 | ☑ | ☐ | ☐ | ☐ |
| A* với h1 | ☐ | ☐ | ☑ | ☐ |
| A* với h2 | ☐ | ☑ | ☐ | ☐ |

**b)** Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

| |
|---|
| Đáp án: 11 (đường đi tối ưu là A - C - E - F - G với chi phí 4 + 2 + 3 + 2 = 11) |

c) Heuristic h1 có admissible không? Có consistent không?
Admissible: ☐ Có ☑ Không (vì h1(A)=12 > h*(A)=11, h1(C)=9 > h*(C)=7)
Consistent: ☐ Có ☑ Không (vì trên cạnh (A,B): |h1(A)-h1(B)| = |12-6| = 6 > 3)

☑ Có ☐ Không

d) Heuristic h2 có admissible không? Có consistent không?
Admissible: ☑ Có ☐ Không (vì mọi nút n đều có h2(n) ≤ h*(n))
Consistent: ☐ Có ☑ Không (vì trên cạnh (A,C): |h2(A)-h2(C)| = |11-6| = 5 > 4, hoặc (E,F): |5-1| = 4 > 3)

☑ Có ☐ Không

## Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

**a)** Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.
☐ Đúng ☑ Sai

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.
☑ Đúng ☐ Sai

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☑ Đúng ☐ Sai

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.
☐ Đúng ☑ Sai

Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

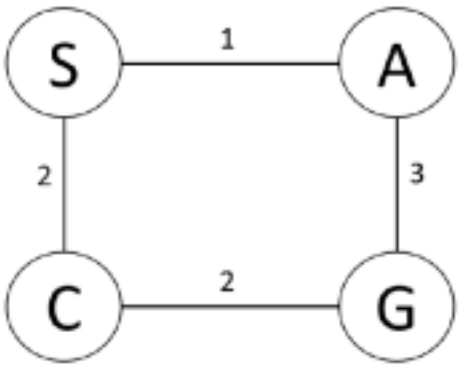
☑ Đúng ☐ Sai

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.

☐ Đúng ☑ Sai

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.
(Giải thích: Chi phí thực tế tối ưu h*(S)=4, h*(A)=3, h*(C)=2, h*(G)=0. Điền h(S)=4, h(A)=0, h(C)=2, h(G)=0. Tất cả đều ≤ h* nên admissible. Tại cạnh S-A: c(S,A)=1 nhưng h(S) - h(A) = 4 - 0 = 4 > 1, vi phạm điều kiện consistency).

**Tìm kiếm và Heuristic trong AI 5 / 5**




| Heuristic Function | |
|---|---|
| **State** | $h(s)$ |
| $S$ | |
| $A$ | |
| $C$ | |
| $G$ | 0 |

| Trạng thái | h(s) |
|---|---|
| S | 4 |
| A | 0 |
| C | 2 |
| G | **0** |

**--- Hết ---**